

VITyarthi - Build Your Own Project

Student Academic Performance and Attendance Management System

Project Report

Course: Problem Solving using Python

Name	Debarghya Chakraborty
Registration No	26BCE11053
Faculty	Devaraju S
Submitted to	VITyarthi

1. Introduction

In college, marks and attendance of students are usually checked by hand or written in different sheets. Adding five subjects and finding the percentage again and again is slow and it is easy to make a mistake. Attendance also has to be checked because a student needs at least 75% to be eligible.

For this project I made a menu-driven Python program that stores student details, marks and attendance and does all the calculations automatically. It gives the total, percentage, grade, pass or fail result, attendance percentage and eligibility. It is a console program written with only basic Python topics from the course: variables, lists, dictionaries, conditions, loops and functions.

2. Problem Statement

Calculating marks, grades and attendance for many students by hand takes time and can give wrong results. There should be a simple program where a teacher or student can enter the data once and get the correct total, percentage, grade, result and attendance status.

Scope: many students, 5 subjects, marks from 0 to 100, pass mark 40%, attendance limit 75%, console based, data is kept only while the program runs. **Target users:** teachers, class coordinators and students.

3. Functional Requirements

ID	Requirement
FR1	Add a student with roll number and name. Same roll number cannot be added twice.
FR2	Enter marks (0-100) for English, Physics, Chemistry, Mathematics and Computer Science.
FR3	Calculate total (out of 500), percentage, grade and PASS/FAIL result.
FR4	Enter total classes and attended classes, then calculate absent classes, attendance percentage and ELIGIBLE / NOT ELIGIBLE.
FR5	Show a full report of one student with marks and attendance.
FR6	Show a table of all students.
FR7	Show a class summary: average, topper, pass/fail count, rank list and low attendance students.
FR8	Menu with 8 options that keeps running until the user chooses Exit.

Grade table used

Percentage	Grade
90 - 100	A+
80 - 89	A
70 - 79	B
60 - 69	C
50 - 59	D
Below 50	F

Result is PASS if percentage is 40 or more, otherwise FAIL. Because of this, a student with 43% gets grade F but the result is still PASS (F is only for below 50, while the pass mark is 40).

4. Non-functional Requirements

Type	How it is handled in this project
Usability	Simple numbered menu, clear messages, report looks neat and easy to read.
Reliability	Wrong input (letters, negative numbers, marks above 100) never crashes the program. It asks again.
Maintainability	Code is divided into 7 small files, each with one job, using functions with simple names. Comments only where needed.
Error handling	Every input goes through validation.py. Missing data is checked before showing a report (for example marks not entered yet).
Performance / resource use	Works instantly and uses very little memory because data is stored in one dictionary.
Scalability	Any number of students can be added because the dictionary grows as needed.

5. System Architecture

The program has one entry file (main.py) that shows the menu. Each menu option calls a function from one of the modules. All modules that take input use validation.py, and all student data is kept in one dictionary called *students* inside student.py.

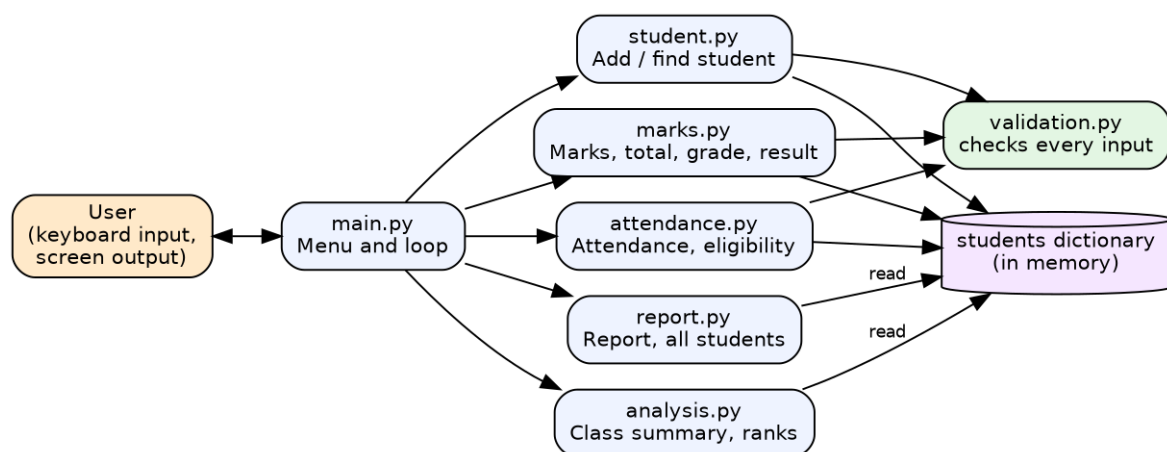


Figure 1: System architecture

6. Design Diagrams

6.1 Use Case Diagram

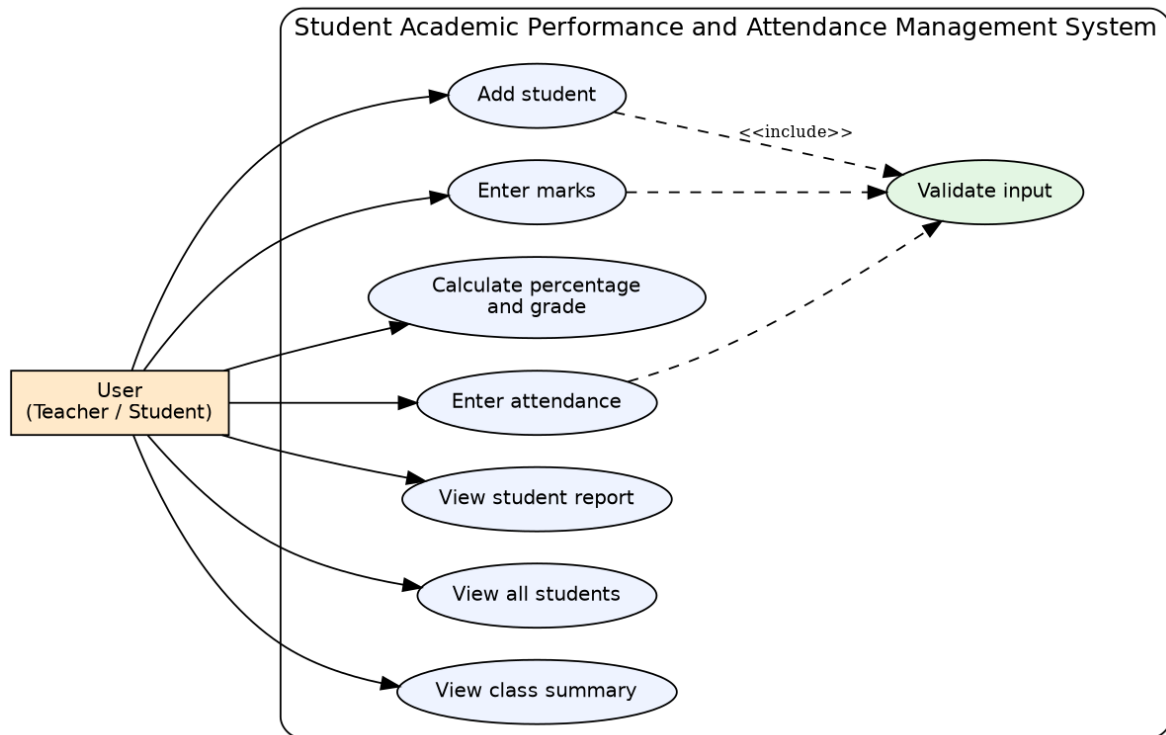


Figure 2: Use case diagram

6.2 Workflow Diagram

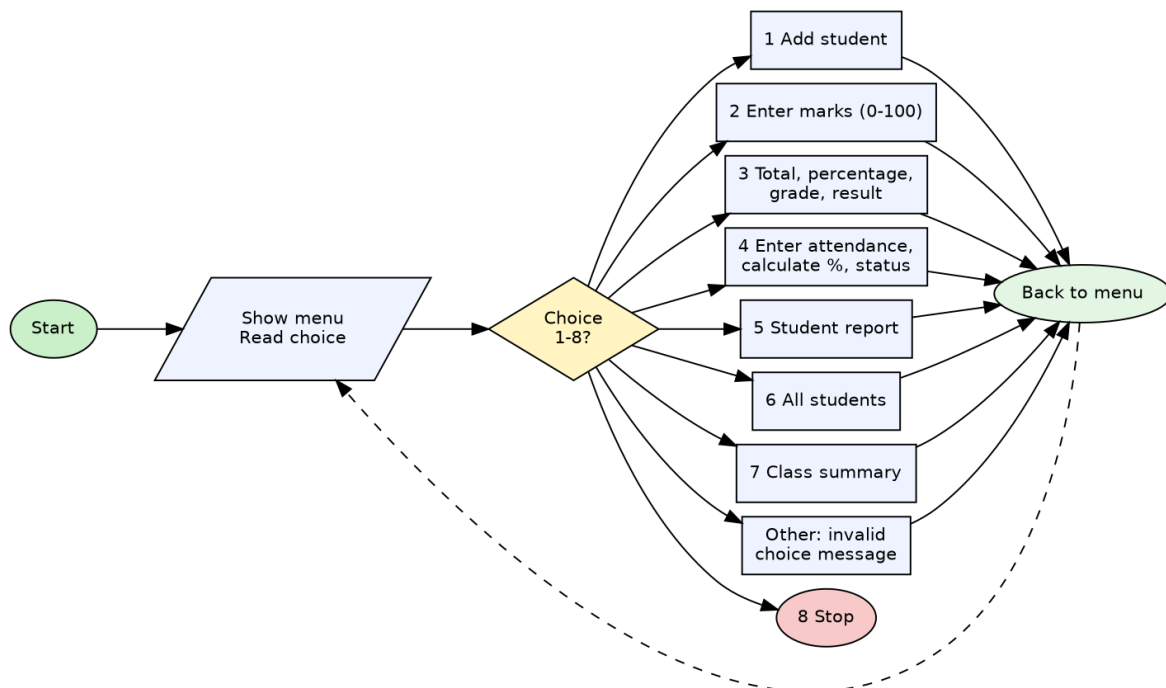


Figure 3: Workflow of the program

6.3 Sequence Diagram (Enter Marks)

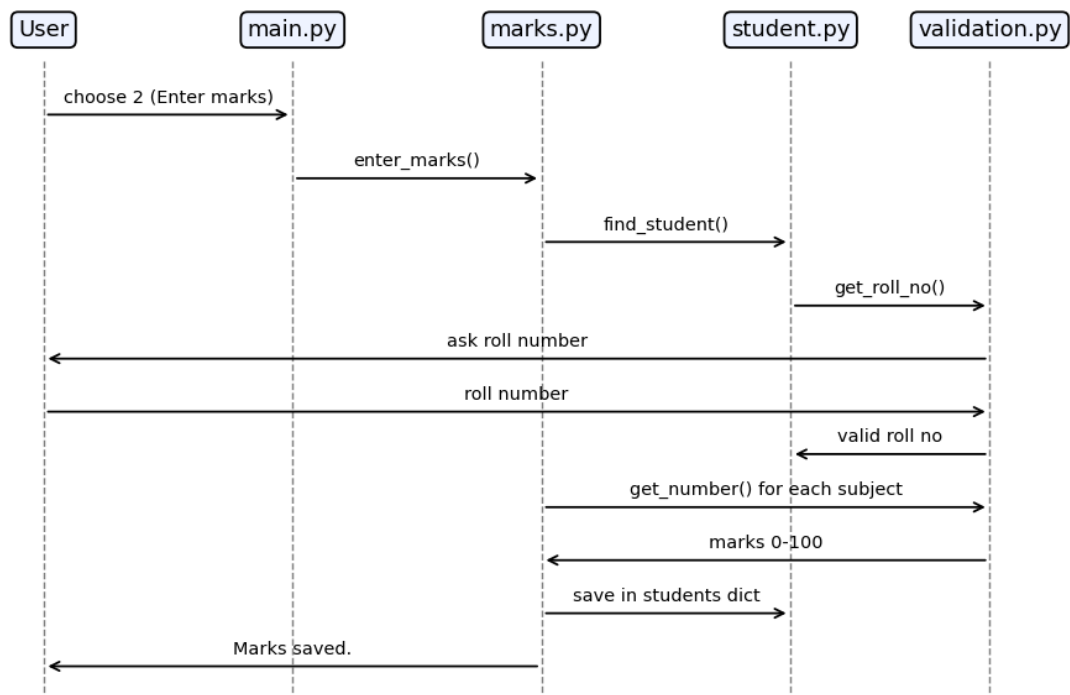


Figure 4: Sequence for option 2 - Enter Marks

6.4 Component Diagram

No classes are used in this project, so a component diagram is given instead of a class diagram. Each file is one component.

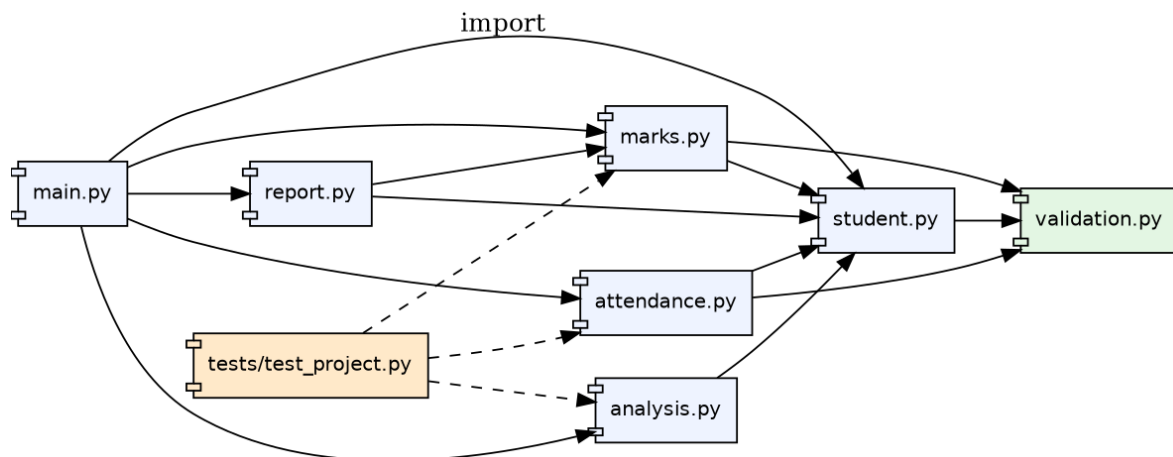


Figure 5: Component diagram

6.5 ER Diagram

Not needed because no database or file storage is used. Student data is stored in a Python dictionary. The structure is:

```
students = {
    "101": {"name": "Rahul", "marks": {"English": 82, ...}, "total": 408,
           "percentage": 81.6, "grade": "A", "result": "PASS",
           "total_classes": 100, "attended": 86, "absent": 14,
           "attendance": 86.0, "status": "ELIGIBLE"}
}
```

7. Design Decisions and Rationale

- **Dictionary for students:** roll number is the key so a student can be found directly without looping through a list.
- **Separate files:** the assignment asks for a modular project. Keeping marks, attendance, report etc. in different files makes each part easy to find and test.
- **No classes:** the course syllabus only covers functions, lists, tuples, sets and dictionaries, so the project stays within these topics.
- **Calculation functions return values:** calculate_percentage, calculate_grade and calculate_attendance only calculate and return, so they can be tested easily without typing input.
- **Roll number stored as text:** it is used only as an ID, not for calculations.
- **No file saving:** kept out to stay simple. It is listed under future enhancements.
- **Class summary added:** this uses counting, summation, finding the maximum and sorting from the syllabus (Fundamental Algorithms and Array Techniques).

8. Implementation Details

The table shows what each file does and where the syllabus topics are used.

File	Purpose	Syllabus topic used
main.py	Menu and while loop, calls the other functions	Control flow (while, if-elif-else), functions
student.py	students dictionary, add_student(), find_student()	Dictionaries
marks.py	enter_marks(), calculate_percentage(), calculate_grade(), get_result(), update_result()	Summation, conditionals, lists
attendance.py	calculate_attendance(), enter_attendance()	Arithmetic expressions, conditionals
report.py	display_report(), view_all_students()	Loops, string formatting
analysis.py	class_summary(), sort_by_percentage()	Counting, maximum, selection sort, swapping values
validation.py	get_number(), get_roll_no(), get_name()	while loop, string methods, validation
tests/test_project.py	5 test functions using assert	Program verification

Algorithm (main steps)

1. Start and show the menu.
2. Read the user's choice.
3. If choice is 1, add a student (roll number and name).
4. If choice is 2, read marks of 5 subjects, each between 0 and 100.
5. If choice is 3, total = sum of marks, percentage = total / 5, then find grade and result.
6. If choice is 4, read total classes and attended classes, find absent, attendance percentage and status.

7. If choice is 5 or 6, print the report or the list of students.
8. If choice is 7, print the class summary.
9. If choice is 8, stop. Otherwise show an error and go back to step 1.

Efficiency

Finding a student is fast because of the dictionary. The rank list uses selection sort, which takes about $n \times n$ steps. This is fine for one class (50 to 100 students).

Complete source code

The full code is also in the GitHub repository. File names are given above each part.

main.py

```
# Student Academic Performance and Attendance Management System
# Name: Debarghya Chakraborty
# Reg No: 26BCE11053

import student
import marks
import attendance
import report
import analysis

def show_menu():
    print()
    print("=" * 40)
    print(" STUDENT MANAGEMENT SYSTEM")
    print("=" * 40)
    print("1. Add Student")
    print("2. Enter Marks")
    print("3. Calculate Percentage and Grade")
    print("4. Enter Attendance")
    print("5. View Student Report")
    print("6. View All Students")
    print("7. Class Summary")
    print("8. Exit")

def main():
    while True:
        show_menu()
        choice = input("Enter your choice (1-8): ").strip()

        if choice == "1":
            student.add_student()
        elif choice == "2":
            marks.enter_marks()
        elif choice == "3":
            marks.update_result()
        elif choice == "4":
            attendance.enter_attendance()
        elif choice == "5":
            report.display_report()
        elif choice == "6":
            report.view_all_students()
        elif choice == "7":
            analysis.class_summary()
        elif choice == "8":
            print("Thank you. Goodbye!")
            break
        else:
            print("Invalid choice. Please enter a number from 1 to 8.")

main()
```

student.py

```

# student.py
# stores all the students in a dictionary
# key = roll number, value = another dictionary with the details

import validation

students = {}

def add_student():
    roll_no = validation.get_roll_no()
    if roll_no in students:
        print("A student with this roll number already exists.")
        return
    name = validation.get_name()
    students[roll_no] = {
        "name": name,
        "marks": {},
        "total": 0,
        "percentage": 0.0,
        "grade": "-",
        "result": "-",
        "total_classes": 0,
        "attended": 0,
        "absent": 0,
        "attendance": 0.0,
        "status": "-",
    }
    print("Student added successfully.")

def find_student():
    """Asks for a roll number and returns the roll number if the student exists."""
    if len(students) == 0:
        print("No students added yet. Please add a student first.")
        return None
    roll_no = validation.get_roll_no()
    if roll_no not in students:
        print("Student not found.")
        return None
    return roll_no

```

marks.py

```

# marks.py
# everything related to marks, percentage, grade and result

import validation
import student

SUBJECTS = ["English", "Physics", "Chemistry", "Mathematics", "Computer Science"]

def enter_marks():
    roll_no = student.find_student()
    if roll_no is None:
        return
    data = student.students[roll_no]
    print("Enter marks for", data["name"], "(0 to 100)")
    for subject in SUBJECTS:
        data["marks"][subject] = validation.get_number(subject + ": ", 0, 100)
    print("Marks saved.")

def calculate_percentage(marks):
    """Returns total and percentage for a marks dictionary."""
    total = 0
    for subject in marks:
        total = total + marks[subject]
    percentage = total / len(SUBJECTS)
    return total, percentage

def calculate_grade(percentage):

```



```

    if percentage >= 90:
        return "A+"
    elif percentage >= 80:
        return "A"
    elif percentage >= 70:
        return "B"
    elif percentage >= 60:
        return "C"
    elif percentage >= 50:
        return "D"
    else:
        return "F"

def get_result(percentage):
    if percentage >= 40:
        return "PASS"
    return "FAIL"

def update_result():
    """Menu option 3 - calculates everything for one student."""
    roll_no = student.find_student()
    if roll_no is None:
        return
    data = student.students[roll_no]
    if len(data["marks"]) < len(SUBJECTS):
        print("Marks are not entered yet. Use option 2 first.")
        return
    total, percentage = calculate_percentage(data["marks"])
    data["total"] = total
    data["percentage"] = percentage
    data["grade"] = calculate_grade(percentage)
    data["result"] = get_result(percentage)
    print("Total      :", total, "/ 500")
    print("Percentage : {:.1f}%".format(percentage))
    print("Grade       :", data["grade"])
    print("Result      :", data["result"])

```

attendance.py

```

# attendance.py
# attendance percentage and eligibility

import validation
import student

def calculate_attendance(total_classes, attended):
    """Returns absent classes, attendance percentage and status."""
    absent = total_classes - attended
    if total_classes == 0:
        percentage = 0.0
    else:
        percentage = attended / total_classes * 100
    if percentage >= 75:
        status = "ELIGIBLE"
    else:
        status = "NOT ELIGIBLE"
    return absent, percentage, status

def enter_attendance():
    roll_no = student.find_student()
    if roll_no is None:
        return
    data = student.students[roll_no]
    total_classes = validation.get_number("Total classes: ", 1, 500)
    attended = validation.get_number("Classes attended: ", 0, total_classes)

    absent, percentage, status = calculate_attendance(total_classes, attended)
    data["total_classes"] = total_classes
    data["attended"] = attended
    data["absent"] = absent

```

```

data["attendance"] = percentage
data["status"] = status
print("Absent      :", absent)
print("Attendance : {:.1f}%".format(percentage))
print("Status      :", status)

```

report.py

```

# report.py
# shows the report of one student and the list of all students

import student
import marks

def display_report():
    roll_no = student.find_student()
    if roll_no is None:
        return
    data = student.students[roll_no]

    if len(data["marks"]) < len(marks.SUBJECTS) or data["total_classes"] == 0:
        print("Marks and attendance both must be entered before viewing the report.")
        print("Use options 2, 3 and 4 first.")
        return

    print()
    print("-" * 40)
    print("          STUDENT REPORT")
    print("-" * 40)
    print()
    print("Roll No:", roll_no)
    print("Name:", data["name"])
    print()
    print("Marks:")
    for subject in marks.SUBJECTS:
        print("{:<17}: {}".format(subject, data["marks"][subject]))
    print()
    print("Total Marks      : {}/500".format(data["total"]))
    print("Percentage       : {:.1f}%".format(data["percentage"]))
    print("Grade            :", data["grade"])
    print("Result           :", data["result"])
    print()
    print("Attendance:")
    print("Total Classes    :", data["total_classes"])
    print("Attended         :", data["attended"])
    print("Absent           :", data["absent"])
    print("Attendance       : {:.1f}%".format(data["attendance"]))
    print("Status           :", data["status"])
    print()
    print("-" * 40)

def view_all_students():
    if len(student.students) == 0:
        print("No students added yet.")
        return
    print()
    print("{:<8}{:<15}{:<8}{:<7}{:<8}{:<14}".format(
        "Roll", "Name", "Perc.", "Grade", "Att.", "Status"))
    print("-" * 60)
    for roll_no in student.students:
        data = student.students[roll_no]
        print("{:<8}{:<15}{:<8}{:<7}{:<8}{:<14}".format(
            roll_no, data["name"],
            "{:.1f}".format(data["percentage"]), data["grade"],
            "{:.1f}".format(data["attendance"]), data["status"]))

```

analysis.py

```

# analysis.py
# small class summary using loops and lists
# (uses the ideas of counting, summation, maximum and sorting)
import student

```

```

def class_summary():
    if len(student.students) == 0:
        print("No students added yet.")
        return

    # only students whose marks are calculated
    done = []
    for roll_no in student.students:
        if student.students[roll_no]["grade"] != "-":
            done.append(roll_no)
    if len(done) == 0:
        print("No student has a calculated result yet (use option 3).")
        return

    total_percentage = 0
    passed = 0
    failed = 0
    topper = done[0]
    for roll_no in done:
        data = student.students[roll_no]
        total_percentage = total_percentage + data["percentage"]
        if data["result"] == "PASS":
            passed = passed + 1
        else:
            failed = failed + 1
        if data["percentage"] > student.students[topper]["percentage"]:
            topper = roll_no

    average = total_percentage / len(done)

    print()
    print("Students with result :", len(done))
    print("Class average       : {:.1f}%".format(average))
    print("Passed                :", passed)
    print("Failed                 :", failed)
    print("Topper                  : {} ({:.1f}%".format(
        student.students[topper]["name"], student.students[topper]["percentage"]))

    print()
    print("Rank list:")
    ranked = sort_by_percentage(done)
    rank = 1
    for roll_no in ranked:
        data = student.students[roll_no]
        print("{} . {} - {:.1f}%".format(rank, data["name"], data["percentage"]))
        rank = rank + 1

    print()
    print("Students with low attendance (below 75%):")
    found = False
    for roll_no in student.students:
        data = student.students[roll_no]
        if data["total_classes"] > 0 and data["status"] == "NOT ELIGIBLE":
            print("-", data["name"], "{:.1f}%".format(data["attendance"]))
            found = True
    if not found:
        print("None")

def sort_by_percentage(roll_list):
    """Selection sort - highest percentage first. Returns a new list."""
    items = roll_list[:]
    for i in range(len(items)):
        best = i
        for j in range(i + 1, len(items)):
            if student.students[items[j]]["percentage"] > student.students[items[best]]["percentage"]:
                best = j
        items[i], items[best] = items[best], items[i]
    return items

```

validation.py

```

# validation.py
# small helper functions to take correct input from the user

```

```

def get_number(message, low, high):
    """Keeps asking until the user enters a whole number between low and high."""
    while True:
        value = input(message)
        if not value.strip().lstrip("-").isdigit():
            print("Please enter a number only.")
            continue
        value = int(value)
        if value < low or value > high:
            print("Number must be between", low, "and", high)
            continue
        return value

def get_roll_no():
    """Roll number must be a positive number like 101."""
    while True:
        roll_no = input("Enter roll number: ").strip()
        if roll_no.isdigit() and int(roll_no) > 0:
            return roll_no
        print("Roll number should contain only digits, for example 101.")

def get_name():
    """Name should not be empty and should have letters only (spaces allowed)."""
    while True:
        name = input("Enter student name: ").strip()
        if name != "" and name.replace(" ", "").isalpha():
            return name.title()
        print("Name should have only letters.")

```

tests/test_project.py

```

# tests/test_project.py
# simple tests - run with: python tests/test_project.py
import os
import sys

sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))

import marks
import attendance
import student
import analysis

def test_percentage():
    data = {"English": 82, "Physics": 76, "Chemistry": 71,
            "Mathematics": 88, "Computer Science": 91}
    total, percentage = marks.calculate_percentage(data)
    assert total == 408
    assert round(percentage, 1) == 81.6

def test_grades():
    assert marks.calculate_grade(95) == "A+"
    assert marks.calculate_grade(90) == "A+"
    assert marks.calculate_grade(89.9) == "A"
    assert marks.calculate_grade(75) == "B"
    assert marks.calculate_grade(60) == "C"
    assert marks.calculate_grade(50) == "D"
    assert marks.calculate_grade(49.9) == "F"

def test_result():
    assert marks.get_result(40) == "PASS"
    assert marks.get_result(39.9) == "FAIL"

def test_attendance():
    absent, percentage, status = attendance.calculate_attendance(100, 86)
    assert absent == 14
    assert percentage == 86
    assert status == "ELIGIBLE"

```

```

assert attendance.calculate_attendance(100, 75)[2] == "ELIGIBLE"
assert attendance.calculate_attendance(100, 74)[2] == "NOT ELIGIBLE"
assert attendance.calculate_attendance(0, 0)[1] == 0.0

def test_sorting():
    student.students.clear()
    for roll, perc in [("1", 60.0), ("2", 90.0), ("3", 75.5)]:
        student.students[roll] = {"percentage": perc}
    assert analysis.sort_by_percentage(["1", "2", "3"]) == ["2", "3", "1"]
    student.students.clear()

all_tests = [test_percentage, test_grades, test_result, test_attendance, test_sorting]
passed = 0
for test in all_tests:
    try:
        test()
        print("PASS -", test.__name__)
        passed = passed + 1
    except AssertionError:
        print("FAIL -", test.__name__)
print(passed, "out of", len(all_tests), "tests passed")

```

9. Screenshots / Results

Sample data: Rahul (roll 101) has marks 82, 76, 71, 88, 91 and attended 86 of 100 classes. Priya Sharma (roll 102) has marks 45, 50, 38, 42, 40 and attended 70 of 100 classes. The output below is copied from the program run. (Replace or add real screenshots of your own terminal in the final PDF if you can.)

Student report (option 5)

```
-----
                        STUDENT REPORT
-----

Roll No: 101
Name: Rahul

Marks:
English      : 82
Physics      : 76
Chemistry    : 71
Mathematics  : 88
Computer Science : 91

Total Marks   : 408/500
Percentage    : 81.6%
Grade        : A
Result       : PASS

Attendance:
Total Classes : 100
Attended      : 86
Absent        : 14
Attendance    : 86.0%
Status        : ELIGIBLE

-----
```

All students (option 6)

Roll	Name	Perc.	Grade	Att.	Status
101	Rahul	81.6	A	86.0	ELIGIBLE
102	Priya Sharma	43.0	F	70.0	NOT ELIGIBLE

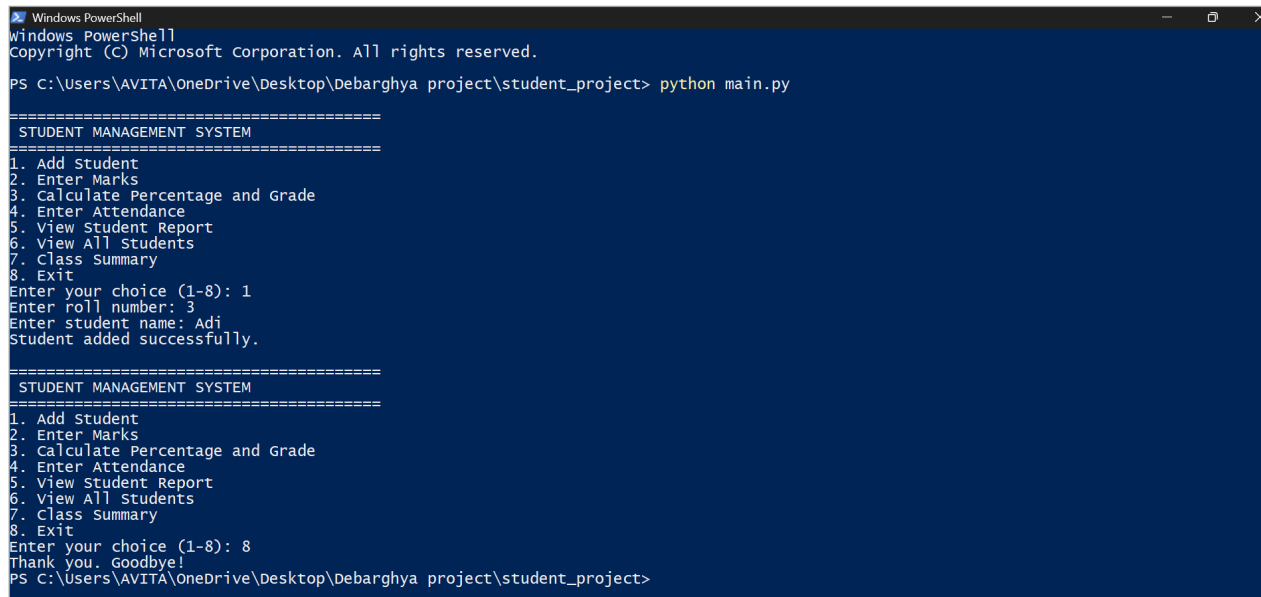
Class summary (option 7)

```
Students with result : 2
Class average        : 62.3%
Passed               : 2
Failed               : 0
Topper               : Rahul (81.6%)

Rank list:
1. Rahul - 81.6%
2. Priya Sharma - 43.0%

Students with low attendance (below 75%):
- Priya Sharma (70.0%)
```

Program run screenshot (add student and exit)



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\AVITA\OneDrive\Desktop\Debarghya project\student_project> python main.py

=====
STUDENT MANAGEMENT SYSTEM
=====
1. Add Student
2. Enter Marks
3. Calculate Percentage and Grade
4. Enter Attendance
5. View Student Report
6. View All Students
7. Class Summary
8. Exit
Enter your choice (1-8): 1
Enter roll number: 3
Enter student name: Adi
Student added successfully.

=====
STUDENT MANAGEMENT SYSTEM
=====
1. Add Student
2. Enter Marks
3. Calculate Percentage and Grade
4. Enter Attendance
5. View Student Report
6. View All Students
7. Class Summary
8. Exit
Enter your choice (1-8): 8
Thank you. Goodbye!
PS C:\Users\AVITA\OneDrive\Desktop\Debarghya project\student_project>
```

Figure 6: Terminal output of main.py (Windows PowerShell)

10. Testing Approach

I tested in two ways. First, a test file (tests/test_project.py) checks the calculation functions using assert. It covers the example from the project (408 marks = 81.6%), all grade boundaries (for example 90 gives A+ and 89.9 gives A), the pass mark of 40, attendance of exactly 75 and 74, zero classes, and the sorting for the rank list.

PASS - test_percentage

PASS - test_grades
PASS - test_result
PASS - test_attendance
PASS - test_sorting
5 out of 5 tests passed

Second, manual testing of wrong inputs:

Test	Input	Expected	Result
Roll number with letters	abc	Message and asks again	OK
Negative roll number	-5	Message and asks again	OK
Name with numbers	R2	Message and asks again	OK
Marks above 100	150	Message and asks again	OK
Negative marks	-3	Message and asks again	OK
Letters in marks	x	'Please enter a number only.'	OK
Attended more than total	total 50, attended 80	Message, asks again	OK
Duplicate roll number	101 twice	'already exists'	OK
Report before marks	option 5 early	Asks to enter marks and attendance first	OK
Invalid menu choice	9	'Invalid choice' message	OK

11. Challenges Faced

- Making the input checking work so the program does not crash when someone types letters. I used a while loop with `isdigit()` to keep asking.
- Deciding how to store many students. First I thought of separate lists for names, marks etc. but a dictionary with roll number as key was easier.
- Keeping the calculation separate from input/output, so that it could be tested.
- The grade table and the pass mark are different (F is below 50 but pass is 40), so a student can have grade F and still PASS. I kept it as given in the requirement and wrote it in the report.

12. Learnings and Key Takeaways

- Breaking a big problem into small functions (top-down design) makes the code much easier to write and fix.
- Nested dictionaries are useful to store records.
- Validation is important. Most of the bugs came from wrong input.
- Writing tests helps to catch mistakes in boundary values like exactly 75% or exactly 90%.
- Using Git and GitHub to save the work in small commits.

13. Future Enhancements

- Save and load student data from a file so it is not lost.
- Add option to edit or delete a student.
- Add more subjects or semesters.
- Add a login for the teacher.
- Make a simple GUI or web version later.

14. References

- Course material and lecture notes of Problem Solving using Python, VITyarthi.
- Python documentation - <https://docs.python.org/3/>
- R. G. Dromey, How to Solve it by Computer (algorithms and program verification).